SineKAN for MNIST Digit Classification

A Kolmogorov-Arnold Network using sinusoidal basis functions (Reinhardt et al., 2024), applied to MNIST.

SineKAN is my choice for this task as the paper showed sine-based activations match or beat B-spline KANs while being significantly faster. Sine functions are also smooth, differentiable everywhere, and map directly to quantum rotation gates (RY/RZ).

References

- Liu et al., "KAN: Kolmogorov-Arnold Networks" (2024), arXiv:2404.19756
- Reinhardt et al., "SineKAN: Kolmogorov-Arnold Networks Using Sinusoidal Activation Functions" (2024), arXiv:2407.04149

Dependencies

In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import time

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
```

```
Device: cuda
```

Configuration

In [2]:
```
BATCH_SIZE = 256
EPOCHS = 10
LR = 1e-3
NUM_FREQUENCIES = 8   # sine/cosine terms per edge
SEED = 42

torch.manual_seed(SEED)
```

Out[2]:
```
<torch._C.Generator at 0x7df4a23281d0>
```

Dataset

MNIST — 60k train / 10k test, 28×28 grayscale → flattened to 784.

In [3]:
```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_data = datasets.MNIST(root='./data', train=True, download=True, transform=tra
test_data = datasets.MNIST(root='./data', train=False, download=True, transform=tra

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True, num_work
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False, num_worke

print(f'Train: {len(train_data):,}  |  Test: {len(test_data):,}')
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 37.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.03MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 8.53MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.68MB/s]
Train: 60,000  |  Test: 10,000
```

SineKAN Layer

Core idea — In a KAN, learnable activation functions live on *edges*, not nodes. Each edge learns a univariate function φ(x) via a truncated Fourier-like series:

$$\varphi(x) = \sum_{k=1}^{K} [a_k \sin(kx) + b_k \cos(kx)]$$

where a's & b's are learnable parameters.

Output node j simply sums its incoming edges:

$$y_j = \sum_i \varphi_{ij}(x_i)$$

In [4]:
```
class SineKANLayer(nn.Module):
    def __init__(self, in_features, out_features, num_frequencies=NUM_FREQUENCIES):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.num_frequencies = num_frequencies

        scale = 1.0 / (in_features * num_frequencies) ** 0.5
        self.a_coeffs = nn.Parameter(torch.randn(out_features, in_features, num_fre
        self.b_coeffs = nn.Parameter(torch.randn(out_features, in_features, num_fre

    def forward(self, x):
        # k = [1, 2, ..., K]
        k = torch.arange(1, self.num_frequencies + 1, device=x.device, dtype=x.dtyp

        # (batch, in, K)
```

```
        kx = x.unsqueeze(-1) * k
        sin_terms = torch.sin(kx)
        cos_terms = torch.cos(kx)

        # Aggregate sum
        y = torch.einsum('oik,bik->bo', self.a_coeffs, sin_terms) + \
            torch.einsum('oik,bik->bo', self.b_coeffs, cos_terms)
        return y
```

SineKAN Model

Architecture: `[784, 64, 10]` — two SineKAN layers with LayerNorm in between.

In [5]:
```
class SineKAN(nn.Module):
    def __init__(self, layer_dims, num_frequencies=NUM_FREQUENCIES):
        super().__init__()
        self.layers = nn.ModuleList()
        self.norms = nn.ModuleList()

        for i in range(len(layer_dims) - 1):
            self.layers.append(SineKANLayer(layer_dims[i], layer_dims[i + 1], num_f
            if i < len(layer_dims) - 2:
                self.norms.append(nn.LayerNorm(layer_dims[i + 1]))

    def forward(self, x):
        for i, layer in enumerate(self.layers):
            x = layer(x)
            if i < len(self.norms):
                x = self.norms[i](x)
        return x
```

MLP Baseline

Same shape `[784, 64, 10]` with Linear + LayerNorm + ReLU for fair comparison.

In [6]:
```
class MLPBaseline(nn.Module):
    def __init__(self, layer_dims):
        super().__init__()
        layers = []
        for i in range(len(layer_dims) - 1):
            layers.append(nn.Linear(layer_dims[i], layer_dims[i + 1]))
            if i < len(layer_dims) - 2:
                layers.append(nn.LayerNorm(layer_dims[i + 1]))
                layers.append(nn.ReLU())
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

Training Loop

In [7]:
```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def train_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images = images.view(images.size(0), -1).to(DEVICE)
        labels = labels.to(DEVICE)
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        correct += (logits.argmax(dim=1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total


@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images = images.view(images.size(0), -1).to(DEVICE)
        labels = labels.to(DEVICE)
        logits = model(images)
        loss = criterion(logits, labels)
        total_loss += loss.item() * images.size(0)
        correct += (logits.argmax(dim=1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, correct / total


def train_model(model, name, epochs=EPOCHS, lr=LR):
    print(f'\n{"="*60}')
    print(f'  Training: {name}  |  Params: {count_parameters(model):,}')
    print(f'{"="*60}')

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

    history = {'train_loss': [], 'test_loss': [], 'train_acc': [], 'test_acc': []}
    start = time.time()

    for ep in range(1, epochs + 1):
        tr_loss, tr_acc = train_epoch(model, train_loader, optimizer, criterion)
        te_loss, te_acc = evaluate(model, test_loader, criterion)
        scheduler.step()
        history['train_loss'].append(tr_loss)
        history['test_loss'].append(te_loss)
        history['train_acc'].append(tr_acc)
```

```
        history['test_acc'].append(te_acc)
        print(f'  Epoch {ep:2d}/{epochs} | '
              f'Train Loss: {tr_loss:.4f} Acc: {tr_acc:.4f} | '
              f'Test Loss: {te_loss:.4f} Acc: {te_acc:.4f}')

    elapsed = time.time() - start
    history['total_time'] = elapsed
    print(f'  Time: {elapsed:.1f}s  |  Final test accuracy: {te_acc:.4f}')
    return history
```

Train SineKAN & MLP

In [8]:
```
ARCH = [784, 64, 10]

kan_model = SineKAN(ARCH, num_frequencies=NUM_FREQUENCIES).to(DEVICE)
kan_hist = train_model(kan_model, 'SineKAN')

mlp_model = MLPBaseline(ARCH).to(DEVICE)
mlp_hist = train_model(mlp_model, 'MLP Baseline')
```

```
============================================================
  Training: SineKAN  |  Params: 813,184
============================================================
  Epoch  1/10 | Train Loss: 0.5140 Acc: 0.8488 | Test Loss: 0.2298 Acc: 0.9301
  Epoch  2/10 | Train Loss: 0.1721 Acc: 0.9488 | Test Loss: 0.1546 Acc: 0.9531
  Epoch  3/10 | Train Loss: 0.1078 Acc: 0.9681 | Test Loss: 0.1298 Acc: 0.9599
  Epoch  4/10 | Train Loss: 0.0684 Acc: 0.9801 | Test Loss: 0.1184 Acc: 0.9645
  Epoch  5/10 | Train Loss: 0.0381 Acc: 0.9900 | Test Loss: 0.0985 Acc: 0.9687
  Epoch  6/10 | Train Loss: 0.0210 Acc: 0.9960 | Test Loss: 0.0997 Acc: 0.9708
  Epoch  7/10 | Train Loss: 0.0106 Acc: 0.9989 | Test Loss: 0.0913 Acc: 0.9725
  Epoch  8/10 | Train Loss: 0.0062 Acc: 0.9998 | Test Loss: 0.0894 Acc: 0.9728
  Epoch  9/10 | Train Loss: 0.0048 Acc: 1.0000 | Test Loss: 0.0902 Acc: 0.9723
  Epoch 10/10 | Train Loss: 0.0043 Acc: 1.0000 | Test Loss: 0.0902 Acc: 0.9728
  Time: 75.6s  |  Final test accuracy: 0.9728

============================================================
  Training: MLP Baseline  |  Params: 51,018
============================================================
  Epoch  1/10 | Train Loss: 0.5110 Acc: 0.8891 | Test Loss: 0.2549 Acc: 0.9304
  Epoch  2/10 | Train Loss: 0.2121 Acc: 0.9415 | Test Loss: 0.1823 Acc: 0.9459
  Epoch  3/10 | Train Loss: 0.1613 Acc: 0.9544 | Test Loss: 0.1525 Acc: 0.9569
  Epoch  4/10 | Train Loss: 0.1304 Acc: 0.9640 | Test Loss: 0.1322 Acc: 0.9623
  Epoch  5/10 | Train Loss: 0.1119 Acc: 0.9695 | Test Loss: 0.1199 Acc: 0.9664
  Epoch  6/10 | Train Loss: 0.0960 Acc: 0.9741 | Test Loss: 0.1115 Acc: 0.9674
  Epoch  7/10 | Train Loss: 0.0853 Acc: 0.9775 | Test Loss: 0.1062 Acc: 0.9692
  Epoch  8/10 | Train Loss: 0.0774 Acc: 0.9803 | Test Loss: 0.1025 Acc: 0.9693
  Epoch  9/10 | Train Loss: 0.0723 Acc: 0.9821 | Test Loss: 0.0991 Acc: 0.9718
  Epoch 10/10 | Train Loss: 0.0695 Acc: 0.9830 | Test Loss: 0.0993 Acc: 0.9709
  Time: 73.2s  |  Final test accuracy: 0.9709
```

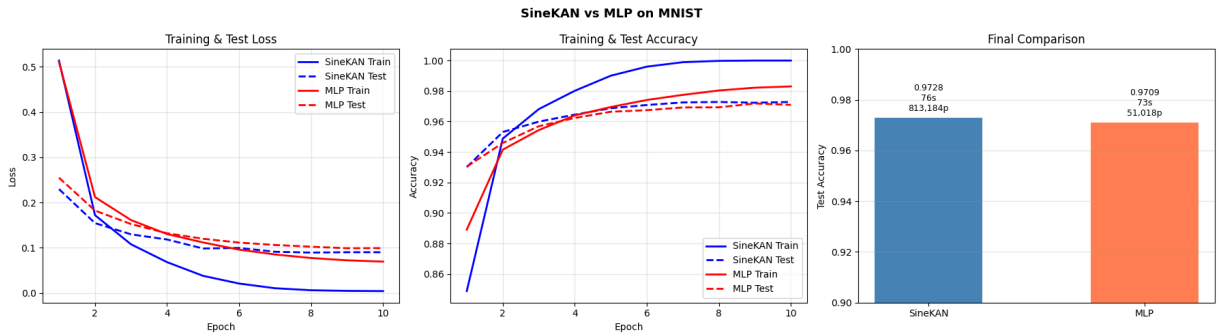
Test Evaluation

In [9]:
```
print('\n' + '-' * 55)
print(f'{"Model":<12} | {"Test Acc":>10} | {"Time (s)":>10} | {"Params":>10}')
print('-' * 55)
print(f'{"SineKAN":<12} | {kan_hist["test_acc"][-1]:>10.4f} | {kan_hist["total_time
print(f'{"MLP":<12} | {mlp_hist["test_acc"][-1]:>10.4f} | {mlp_hist["total_time"]:>
print('-' * 55)
```

```
-------------------------------------------------------
Model        |   Test Acc |   Time (s) |     Params
-------------------------------------------------------
SineKAN      |     0.9728 |       75.6 |    813,184
MLP          |     0.9709 |       73.2 |     51,018
-------------------------------------------------------
```

Result Plots

In [10]:
```
ep_range = range(1, EPOCHS + 1)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('SineKAN vs MLP on MNIST', fontsize=13, fontweight='bold')

# Loss
axes[0].plot(ep_range, kan_hist['train_loss'], 'b-', label='SineKAN Train', lw=2)
axes[0].plot(ep_range, kan_hist['test_loss'], 'b--', label='SineKAN Test', lw=2)
axes[0].plot(ep_range, mlp_hist['train_loss'], 'r-', label='MLP Train', lw=2)
axes[0].plot(ep_range, mlp_hist['test_loss'], 'r--', label='MLP Test', lw=2)
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].set_title('Training & Test Loss')
axes[0].legend(); axes[0].grid(True, alpha=0.3)

# Accuracy
axes[1].plot(ep_range, kan_hist['train_acc'], 'b-', label='SineKAN Train', lw=2)
axes[1].plot(ep_range, kan_hist['test_acc'], 'b--', label='SineKAN Test', lw=2)
axes[1].plot(ep_range, mlp_hist['train_acc'], 'r-', label='MLP Train', lw=2)
axes[1].plot(ep_range, mlp_hist['test_acc'], 'r--', label='MLP Test', lw=2)
axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].set_title('Training & Test Accuracy')
axes[1].legend(); axes[1].grid(True, alpha=0.3)

# Bar comparison
models = ['SineKAN', 'MLP']
test_accs = [kan_hist['test_acc'][-1], mlp_hist['test_acc'][-1]]
times = [kan_hist['total_time'], mlp_hist['total_time']]
params = [count_parameters(kan_model), count_parameters(mlp_model)]

bars = axes[2].bar(models, test_accs, color=['steelblue', 'coral'], width=0.5)
axes[2].set_ylabel('Test Accuracy'); axes[2].set_title('Final Comparison')
axes[2].set_ylim(0.9, 1.0)
for bar, acc, t, p in zip(bars, test_accs, times, params):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
                 f'{acc:.4f}\n{t:.0f}s\n{p:,}p', ha='center', va='bottom', fontsize
axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



Result

- SineKAN trains and converges on MNIST without issues.
- The sine/cosine basis provides smooth, well-behaved gradients.
- LayerNorm + AdamW stabilize training nicely.

Limitations

- SineKAN is slower per epoch , each edge computes K sine/cosine evaluations instead of a single multiply-add.
- More parameters: 2*K params per edge vs 1 weight in MLP. For `[784, 64, 10]` with K=8: SineKAN ≈ 813K params vs MLP ≈ 51K.
- For a simple task like MNIST, the extra expressiveness doesn't dramatically outperform a well-tuned MLP.

Takeaway

KANs shine where interpretable, smooth function approximation matters (scientific computing, symbolic regression). For pure classification, MLPs remain hard to beat on cost-performance. But KANs offer a fundamentally different inductive bias that may be valuable in other domains.